

Cinema Hub



Web Service



Team:

- Shahd Sayed Ahmed Abady 20230285
- Rawan Rabie Abass Mohamed 20230215
- Salma Essam Hamdy Sayed 20230254
- Sara Hatem Mohamed Ahmed 20230244
- Mennatallah Walid Ahmed 20230591
- Menna Shehata Helal Ahmed 20230593

Table of Contents

2.3	Stakeholders	5
7.1	Admin Role and Authorization	6
7.2	Manager Role and Authorization	7
1.	Access and Authentication	7
2.	Operational Management Permissions.....	7
3.	Oversight and Reporting Responsibilities	8
7.3	Customer Role and Authorization	9
1.	Registration and Authentication	9
2.	Booking and Reservation Permissions.....	9
4.1	Authentication Service (Auth Service).....	11
4.2	Movie Service	12
4.3	Booking Service.....	14
4.4	Payment Service.....	17
4.5	API Gateway Service.....	19
4.1	Auth Service (ID: FR-AUTH) Functional Requirements	20
4.2	Movie Service (ID: FR-MOV) Functional Requirements.....	22
4.3	Payment Service (ID: FR-PAY) Functional Requirements.....	24
4.4	Booking Service (ID: FR-BKG) Functional Requirements	25
5.1	Microservices Scalability Evolution.....	29
1)	Use Case Diagram:	36
2)	Activity Diagrams:	37
-	Admin Activity Diagram:.....	37
-	Manager Activity Diagram:.....	39
-	Customer Activity Diagram:	41
3)	Class Diagrams:	43
-	Main Class:.....	43
-	Authentication Class:	45
-	Movie Class:.....	46
-	Payment Class:	47
-	Booking Class:	48
4)	Sequence Diagrams:	49
-	Customer Booking Sequence:	Error! Bookmark not defined.
-	Manager Add Movie Sequence:	50
5)	Entity Relationship Diagram (ERD):	51

1. Introduction

1.1 Purpose

The purpose of this document is to provide a comprehensive and detailed description of the **Cinema Hub** system. It outlines the functional and non-functional requirements, system architecture, and design constraints. This document serves as a guide for developers, testers, and stakeholders throughout the software development life cycle (SDLC).

1.2 Scope

Cinema Hub is a web-based movie ticket booking application designed using a Microservices architecture. The system allows users to browse movies, check cinema availability, select specific seat types (Standard, VIP, Recliner), choose hall types (AC/Non-AC), and purchase snacks.

The backend is powered by Spring Boot, while the frontend is built with React.js. Kafka is utilized for real-time seat synchronization, and the entire system is Dockerized

1.3 Glossary and Abbreviations

List and explain any terms, acronyms, or abbreviations used in this document.

JWT	JSON Web Token (for authentication)
AOP	Aspect-Oriented Programming (used for logging and security)
Software Requirements Specification (SRS)	A document that completely describes all of the functions of a proposed system and the constraints under which it must operate. For example, this document.
Kafka	Distributed event store and stream-processing platform
Standard/VIP/Recliner	Seat categories available for booking.
Docker	Air conditioning options inside the cinema hall. AC: with air conditioner, Non-AC: without air conditioner
AC / Non-AC	Air conditioning options inside the cinema hall. AC: with air conditioner, Non-AC: without air conditioner
Microservice	An architectural style that structures an application as a collection of services

2. System Overall Description

2.1 Product perspective

Cinema Hub is an independent platform that interacts with multiple cinema databases to aggregate showtimes. It uses a decentralized architecture where each module (Booking, Payment, Movie, Auth) has its own database.

2.2 System Architecture (*microservices*)

The system is divided into five main microservices

Auth Service	Movie Service	Booking Service	Payment Service
Handles registration, login, and Role-Based Access Control (RBAC)	Manages movie metadata, posters, and cinema information	Handles the core logic of reservations, seat types, and Kafka integration	Processes transactions and handles mock payment gateways

API Gateway Service

The API Gateway Service acts as the single entry point for all client requests within the Cinema Hub system.

2.3 Stakeholders

Stakeholder	Role
Guest User	Can browse movies but cannot book
Registered User	A user with an account who can browse movies and books tickets, offers and food.
Manager	Controls specific cinema branches, showtimes, and seat availability.
Admin	A user who has access to full system control, managing movies, cinemas, and users

3. User Roles, Permissions, and Responsibilities

The Cinema Hub system defines multiple user roles with different permissions and operational responsibilities. Access control is implemented using JWT-based authentication and Role-Based Access Control (RBAC) mechanisms to ensure secure and authorized interaction with system services.

7.1 Admin Role and Authorization

The Admin role has full system control and is responsible for managing users, monitoring system activities, and maintaining cinema operations across all services.

1. Access and Authentication

The administrative process begins with a secure authentication workflow handled by the Authentication Service.

System Workflow:

- The Admin accesses the admin dashboard and enters login credentials.
- The Authentication Service validates the provided credentials.
- Upon successful authentication, the system generates a JWT authentication token.
- The system grants access to authorized administrative functionalities.
- If authentication fails, the system returns an access error message.

2. Monitoring and Reporting Responsibilities

The Admin is responsible for monitoring system health and reviewing operational reports.

Reporting Dashboard

Admin can:

- Access booking reports from the Booking Service.
- Access payment reports from the Payment Service.
- Analyze overall business and operational performance.

System Monitoring

Admin can:

- Monitor system activity logs.
- Review authentication and access logs.
- Monitor booking and payment operations for auditing and security purposes.

7.2 Manager Role and Authorization

The Manager role focuses on operational management and cinema content maintenance within assigned cinema branches.

1. Access and Authentication

The Manager accesses the system through a secure authentication process managed by the Authentication Service.

System Workflow:

- The Manager opens manager's dashboard.
- Login credentials are submitted to the Authentication Service.
- The system validates manager credentials and permissions.
- Upon successful authentication, the system generates a JWT authentication token.
- Access is granted to manager-specific functionalities.
- Invalid credentials result in an authentication error.

2. Operational Management Permissions

After authentication, the Manager can perform operational and scheduling activities related to cinema management.

Movie Management

The Manager can:

- Add movie records.
- Edit existing movie information.
- Delete movie entries.
- Save movie updates through the Movie Service.

Showtime Scheduling

The Manager can:

- Create and manage showtime schedules.
- Update or cancel showtimes.
- Validate schedules against existing reservations and hall availability to prevent conflicts.

Hall and Seat Configuration

The Manager can:

- Configure cinema halls and seating layouts.
- Manage seat capacities and arrangements.
- Save hall configuration updates within the system.

3. Oversight and Reporting Responsibilities

The Manager has access to operational summaries and monitoring features related to cinema activities.

Booking Overview

The Manager can:

- View booking summaries from the Booking Service.
- Monitor reservation activity and seat occupancy status.

Financial Monitoring

The Manager can:

- Retrieve payment summaries from the Payment Service.
- Monitor payment activities and financial performance indicators.

7.3 Customer Role and Authorization

The Customer role focuses on the end-user booking experience, including movie browsing, seat reservation, payment processing, and ticket management.

1. Registration and Authentication

Customers interact with the system through registration and secure authentication workflows managed by the Authentication Service.

System Workflow:

- Customers access the Cinema Hub platform.
- New users can create an account through the registration process.
- The Authentication Service validates registration data and creates the user account.
- Existing users can log in using their credentials.
- Upon successful authentication, the system generates a JWT authentication token.
- Authorized customers gain access to booking and payment functionalities.

2. Booking and Reservation Permissions

Customers can browse movies and perform booking-related operations through the Movie Service and Booking Service.

Movie Browsing

Customers can:

- Browse available movies.
- View movie details, cinema locations, and showtimes.
- Search for movies and available sessions.

Reservation Process

Customers can:

- Select movies, showtimes, and preferred seats.
- Request seat reservations through the Booking Service.
- Receive booking summaries and pricing details before confirmation.

Real-Time Seat Locking

The system uses Kafka-based real-time event processing to prevent multiple users from reserving the same seat simultaneously.

System Behavior:

- When a customer selects a seat, the Booking Service publishes a seat-selection event to Kafka.
- The system temporarily locks the selected seat.
- Locked seats remain reserved for a limited duration during the payment process.
- If the booking is not completed within the allowed period, the system automatically releases the seat.

3. Payment and Ticket Management

Customers can complete payments and manage their booking activities through the Payment Service and Booking Service.

Payment Operations

Customers can:

- Select a payment method.
- Submit payment details securely.
- Complete payment transactions through the Payment Service.

Booking Confirmation

If payment is successful:

- The booking status is updated to CONFIRMED.
- The system generates a digital ticket with a unique identifier and QR code.
- The customer receives booking confirmation details.

Post-Booking Operations

Customers can:

- View booking history.
- Download electronic tickets.
- Cancel bookings when allowed by system policies.
- Receive refunds for eligible cancelled bookings.
- Release reserved seats automatically after cancellation.

4. Microservices Architecture

The Cinema Hub system follows a Microservices Architecture in which the application is divided into multiple independently deployable services. Each service is responsible for a specific business domain and communicates with other services through REST APIs and event-driven messaging using Apache Kafka.

4.1 Authentication Service (Auth Service)

- The Authentication Service is responsible for managing user identity, authentication, authorization, and access control across the entire Cinema Hub system.
- The service handles user registration, login operations, JWT token generation and validation, role management, and secure access to protected system resources.

Core Responsibilities

User Registration

The service allows new users to create accounts by submitting required personal information such as:

- Name
- Email address
- Password
- Phone number

The system validates input data, checks for duplicate accounts, and securely encrypts passwords before storing them in the database.

User Authentication

The service validates login credentials and authenticates users securely.

After successful authentication:

- A JWT authentication token is generated.
- User role information is embedded within the token.
- Access is granted to authorized resources.

Authorization and Role-Based Access Control (RBAC)

The Auth Service manages user roles and permissions including:

- Admin
- Manager
- Customer

The system verifies whether a user has permission to perform specific operations before granting access.

Token Validation

The service validates JWT tokens by:

- Verifying token signatures
- Checking expiration dates
- Validating embedded user information

User and Permission Management

The service allows administrators to:

- Manage user accounts
- Assign roles
- Update permissions
- Control access privileges

Security Features

The Authentication Service implements several security mechanisms including:

- JWT-based authentication
- Password encryption
- Role-Based Access Control (RBAC)
- Secure session validation
- Access restriction for protected APIs

4.2 Movie Service

- The Movie Service is responsible for managing all movie-related and cinema-related information within the Cinema Hub system.
- The service handles movies, cinema branches, halls, seating configurations, and movie showtimes.

Core Responsibilities

Movie Management

The service allows administrators and managers to:

- Add new movies
- Update movie information
- Delete existing movies
- Retrieve movie details and movie lists

Movie information includes:

- Title
- Genre
- Language
- Duration
- Description
- Posters and media assets

Cinema Management

The service manages cinema branches and related operational information such as:

- Cinema names
- Locations
- Hall assignments

Hall Configuration Management

The service manages cinema halls and seating arrangements.

Responsibilities include:

- Creating hall configurations
- Defining hall capacities
- Configuring seat layouts
- Managing seat categories:
 - Standard
 - VIP
 - Recliner

Showtime Scheduling

The service manages movie scheduling operations including:

- Creating showtimes
- Updating schedules
- Cancelling showtimes
- Assigning movies to halls

Each showtime contains:

- Start time
- End time
- Pricing information
- Hall assignment

Schedule Validation

The Movie Service validates schedules to ensure:

- No overlapping showtimes
- Hall availability consistency
- Conflict-free scheduling operations

Seat Availability Management

The service maintains seat states including:

- Available
- Reserved
- Occupied

Seat availability is synchronized with the Booking Service during reservation operations.

4.3 Booking Service

- The Booking Service is the core business service responsible for handling ticket reservations, seat management, booking workflows, ticket generation, and customer reservation operations.
- The service coordinates with the Movie Service and Payment Service to complete booking transactions successfully.

Core Responsibilities

Booking Creation

The service allows customers to:

- Create bookings
- Select movies and showtimes
- Reserve seats temporarily
- Manage booking details

New bookings are initially created with a PENDING status until payment is completed.

Seat Reservation Management

The Booking Service handles:

- Seat availability validation
- Temporary seat reservation
- Seat release after cancellation or timeout
- Permanent seat assignment after payment confirmation

Real-Time Seat Locking Using Kafka

The system uses Apache Kafka to prevent concurrent seat reservation conflicts.

System Workflow

- When a customer selects a seat, the Booking Service publishes a seat-selection event to a Kafka topic.
- Kafka consumers process the event and temporarily lock the seat.
- Locked seats remain unavailable for a predefined duration.
- If payment is not completed within the allowed time, the system automatically releases the seat through lock-expiration events.

This mechanism ensures:

- Real-time synchronization
- High concurrency handling
- Prevention of duplicate bookings

Food and Offer Management

The service allows customers to:

- Add food items to bookings
- Apply promotional offers and discounts
- Calculate booking totals dynamically

Booking Confirmation

After successful payment verification:

- Booking status changes to CONFIRMED
- Reserved seats become permanently assigned
- Digital ticket generation is triggered

Ticket Generation

The service generates electronic tickets containing:

- Unique ticket identifiers
- QR codes
- Showtime and seat information

Booking Cancellation and Refund Coordination

The service allows customers to:

- Cancel bookings
- Trigger refund operations through the Payment Service
- Release reserved seats automatically

Booking History and Reporting

The service provides:

- Customer booking history
- Booking summaries
- Reservation statistics
- Operational reporting data

4.4 Payment Service

- The Payment Service is responsible for processing payment transactions, validating payment operations, handling refunds, and managing transaction-related events.
- The service ensures secure and reliable financial processing within the Cinema Hub system.

Core Responsibilities

Payment Processing

The service processes payment requests submitted during booking confirmation.

Responsibilities include:

- Receiving payment information
- Validating payment data
- Communicating with payment gateways
- Generating transaction records
- Updating payment statuses

Payment statuses may include:

- Pending
- Successful
- Failed
- Refunded

Payment Verification

The service verifies payment completion by:

- Checking transaction status
- Validating payment provider responses
- Confirming successful financial transactions

Webhook Handling

The Payment Service processes webhook notifications from external payment providers.

The service:

- Validates webhook authenticity
- Processes payment events
- Updates payment records
- Logs transaction activities

Refund Processing

The service supports refund operations for cancelled bookings.

Responsibilities include:

- Validating refund eligibility
- Sending refund requests
- Updating payment status
- Recording refund transactions

Payment Event Logging

The service records all payment-related events for:

- Auditing
- Monitoring
- Security tracking
- Financial reporting

Payment Reporting and Analytics

The Payment Service generates:

- Payment summaries
- Transaction statistics
- Refund reports
- Financial activity reports

These reports support both administrators and managers in monitoring system financial performance.

Security Features

The Payment Service implements several security mechanisms including:

- Secure payment validation
- Webhook signature verification
- Encrypted transaction handling
- Payment event auditing
- Fraud prevention validation mechanisms

4.5 API Gateway Service

- Instead of communicating directly with individual microservices, clients interact with the API Gateway, which routes requests to the appropriate internal services.
- The API Gateway simplifies client communication, improves security, and centralizes cross-cutting concerns such as authentication, request routing, and monitoring.

Core Responsibilities

- Request Routing
- The API Gateway receives incoming client requests and forwards them to the appropriate microservice, including:
 - Auth Service
 - Movie Service
 - Booking Service
 - Payment Service

This routing mechanism hides internal service details from clients and provides a unified system interface.

4. Functional requirements

This section provides a detailed breakdown of the functional capabilities of the **Cinema Hub** system. Each requirement is defined through **System Requirement Specifications (SRS)**—describing the technical logic and constraints. Specifications are presented using structured natural language to ensure clarity for developers and stakeholders.

4.1 Auth Service (ID: FR-AUTH) Functional Requirements

4.1.1 User Registration (FR-AUTH-01)

Function: registerUser(user: User)

Description: The system shall allow users to create accounts using personal information such as name, email, password, and phone number.

System Behavior:

- The system validates the email format and password strength.
- The system verifies that the email is not already registered.
- Passwords are encrypted before storage.
- The system creates and stores the new user account securely.

Priority: High

4.1.2 User Authentication and JWT Management (FR-AUTH-02)

Function: loginUser(email, password)

Description: The system shall authenticate users and generate secure JWT authentication tokens.

System Behavior:

- The system validates user credentials against stored data.
- If credentials are valid, the system generates a signed JWT authentication token containing user role information.
- If credentials are invalid, authentication is rejected.
- The system validates JWT tokens before granting access to protected resources.

Priority: High

4.1.3 Role and Permission Management (FR-AUTH-03)

Function: assignRole(user, role)

Description: The system shall manage user roles and permissions using Role-Based Access Control (RBAC).

System Behavior:

- Administrators can assign or update user roles.
- The system verifies permissions before allowing protected operations.
- Unauthorized access requests are rejected.

Priority: High

4.1.4 User Management (FR-AUTH-04)

Function: manageUsers()

Description: The system shall allow administrators to manage user accounts.

System Behavior:

- Administrators can view, update, and delete user accounts.
- The system securely stores updated user information.

Priority: Medium

4.1.5 Edit Profile (FR-AUTH-05)

Function: editProfile()

Description: The system shall allow users to update their personal information.

System Behavior:

- The system validates updated profile information.
- The updated information is stored securely in the database.

Priority: Low

4.1.6 Logout User (FR-AUTH-06)

Function: logoutUser(token)

Description: The system shall securely terminate authenticated user sessions.

System Behavior:

- The system invalidates the JWT authentication token.
- User access to protected resources is terminated securely.

Priority: Medium

4.2 Movie Service (ID: FR-MOV) Functional Requirements

4.2.1 Movie Management (FR-MOV-01)

Function:

addMovie(movie: Movie)

updateMovie(movie: Movie)

deleteMovie(movieId: Long)

Description: The system shall manage movie records including creation, updates, and deletion operations.

System Behavior:

- The system validates movie information such as title, duration, and genre.
- The system creates unique identifiers for new movie records.
- The system updates modified movie attributes.
- The system prevents deletion when active showtimes are associated with the movie.
- The system stores all movie data persistently.

Priority: High

4.2.2 Retrieve Movies and Details (FR-MOV-02)

Function:

getMovies()

getMovieDetails(movieId: Long)

Description: The system shall retrieve movie lists and detailed movie information.

System Behavior:

- The system fetches movie records from the database.
- The system supports filtering and sorting operations.
- The system retrieves complete movie details including showtimes and cinema information.

Priority: High

4.2.3 Cinema and Hall Management (FR-MOV-03)

Function:

addCinema()

updateCinema()

saveHallConfiguration(hall: Hall)

Description: The system shall manage cinema branches and hall configurations.

System Behavior:

- The system creates and updates cinema information.
- The system manages hall capacities and seating arrangements.
- The system ensures that seat configurations do not exceed hall capacity.
- The system stores hall configuration data persistently.

Priority: High

4.2.4 Showtime Management (FR-MOV-04)

Function:

createShowtime()
updateShowtime()
cancelShowtime()

Description: The system shall manage movie showtimes within cinema halls.

System Behavior:

- The system creates showtimes with start time, end time, and pricing information.
- The system links showtimes to movies and halls.
- The system updates or cancels showtimes when required.
- The system stores all scheduling information persistently.

Priority: High

4.2.5 Showtime Schedule Validation (FR-MOV-05)

Function: validateSchedule(showtime: Showtime)

Description: The system shall validate movie schedules to prevent hall scheduling conflicts.

System Behavior:

- The system checks existing showtimes within the same hall.
- The system ensures that no time overlap exists between showtimes.
- The system returns:
 - true → valid schedule
 - false → conflict detected

Priority: High

4.2.6 Seat Availability Management (FR-MOV-06)

Function:

markAvailable()
markReserved()

Description: The system shall manage seat availability states for movie showtimes.

System Behavior:

- The system assigns seats to hall configurations.
- The system updates seat status between AVAILABLE and RESERVED.
- The system maintains consistency between booking operations and seat states.

Priority: High

4.3 Payment Service (ID: FR-PAY) Functional Requirements

4.3.1 Process Payment (FR-PAY-01)

Function: processPayment(payment: Payment)

Description: The system shall process payment transactions for bookings using supported payment methods.

System Behavior:

- The system receives and validates payment information.
- The system communicates with external payment providers.
- The system creates transaction records with unique transaction identifiers.
- The system updates payment status (Pending → Success/Failed).
- The system stores payment and transaction data securely.
- The system verifies successful transaction completion before confirming payment.

Priority: High

4.3.2 Handle Payment Webhooks (FR-PAY-02)

Function: handleWebhook(payload: WebhookPayload)

Description: The system shall process webhook notifications from external payment providers.

System Behavior:

- The system validates webhook authenticity and payload integrity.
- The system processes payment event types such as success, failure, and refunds.
- The system updates related payment and transaction records.
- The system records payment events for auditing and monitoring purposes.
- Invalid or tampered payloads are rejected.

Priority: High

4.3.3 Verify Payment (FR-PAY-03)

Function: verifyPayment(paymentId: Long): boolean

Description: The system shall verify the status and validity of a payment transaction.

System Behavior:

- The system retrieves the payment using paymentId.
- The system checks the transaction status from the payment provider.
- The system validates that the payment is completed successfully.
- The system returns:
 - true → if payment is valid and successful
 - false → if payment failed or is invalid

Priority: High

4.3.4 Refund Payment (FR-PAY-04)

Function: refundPayment(paymentId: Long)

Description: The system shall process refunds for eligible completed payments.

System Behavior:

- The system retrieves and validates payment information.
- The system verifies refund eligibility.
- The system sends refund requests to the payment provider.
- The system updates payment status to REFUNDED.
- Refund transaction details are stored securely.

Priority: High

4.3.5 Payment Reporting and Summary (FR-PAY-05)

Function: getPaymentSummary()

Description: The system shall generate summaries and reports for payment activities.

System Behavior:

- The system aggregates payment records and statistics.
- The system calculates successful, failed, and refunded transactions.
- The system returns structured payment summary reports.

Priority: Medium

4.4 Booking Service (ID: FR-BKG) Functional Requirements

4.4.1 Seat Availability Validation (FR-BKG-01)

Function: checkSeatAvailability(showtimeId: Long, seatIds: List)

Description: The system shall verify whether selected seats are available before booking confirmation.

System Behavior:

- The system retrieves seat information for the selected showtime.
- The system validates availability status for each seat.
- The system returns:
 - true → all seats are available
 - false → one or more seats are unavailable

Priority: High

4.4.2 Booking Creation and Seat Reservation (FR-BKG-02)

Function:

createPendingBooking(customerId: Long, showtimeId: Long)

addSeats(bookingId: Long, seatIds: List)

Description: The system shall create bookings and reserve selected seats temporarily before payment confirmation.

System Behavior:

- The system creates a booking with PENDING status.
- The system validates selected seat availability.
- The system reserves selected seats temporarily.
- The system links reserved seats to the booking entity.

Priority: High

4.4.3 Food Items and Discount Management (FR-BKG-03)

Function:

addFoodItems()

applyOffer()

Description: The system shall allow customers to add food items and apply discounts to bookings.

System Behavior:

- The system validates food item availability.
- The system applies promotional offers and discounts.
- The system updates booking totals dynamically.

Priority: High

4.4.4 Calculate Booking Total (FR-BKG-04)

Function: calculateTotalPrice(bookingId: Long)

Description: The system shall calculate the final booking cost.

System Behavior:

- The system calculates seat prices and food costs.
- The system applies discounts and taxes.
- The system returns the final booking total.

Priority: High

4.4.5 Confirm or Cancel Booking (FR-BKG-05)

Function:

confirmBooking(bookingId: Long)

cancelBooking(bookingId: Long)

Description: The system shall confirm or cancel bookings based on payment status and customer actions.

System Behavior:

- The system validates payment completion before confirmation.
- Confirmed bookings are updated to CONFIRMED status.
- Cancelled bookings are updated to CANCELLED status.
- The system releases reserved seats after cancellation.
- Refund processing is triggered when applicable.

Priority: High

4.4.6 Ticket Generation and Download (FR-BKG-06)

Function:

generateTicket(bookingId: Long)

downloadTicket()

Description: The system shall generate and provide downloadable digital tickets for confirmed bookings.

System Behavior:

- The system generates unique ticket identifiers and QR codes.
- The system links tickets to booking records.
- Tickets are exported as downloadable PDF files.

Priority: High

4.4.7 Booking History and Reporting (FR-BKG-07)

Function:

getCustomerBookings(customerId: Long)

generateSummary()

Description: The system shall provide booking history and booking summary reports.

System Behavior:

- The system retrieves booking records for specific customers.
- The system generates booking statistics and operational summaries.
- The system calculates confirmed and cancelled booking counts.

Priority: Medium

4.4.8 Real-Time Seat Locking via Kafka (FR-BKG-08)

Function: handleSeatSelection(seatId: Long, bookingId: Long)

Description: The system shall prevent concurrent seat reservations using Kafka-based real-time event processing.

System Behavior:

- When a customer selects a seat, the Booking Service publishes a “SeatSelected” event to a Kafka topic.
- Consumer services listen to the topic and temporarily lock the selected seat.
- The seat remains locked for 10 minutes during the payment process.
- If booking is not completed within the allowed duration, a “LockExpired” event is triggered.
- The system automatically releases expired seat locks.

Priority: High

5. System Evolution

The **Cinema Hub system** is expected to evolve over time due to several influencing factors:

5.1 Microservices Scalability Evolution

- The system shall be designed to support future expansion of microservices without affecting existing services.

5.2 Kafka Event-Driven Expansion

- The system shall support future enhancement of real-time communication using Kafka event streaming.

5.3 Containerization with Docker

- The system shall support containerized deployment using Docker to ensure portability and consistency across environments.

5.4 Hardware Advancements

- Future improvements in server infrastructure and cloud computing may enable:
- Faster processing of booking requests
- Improved handling of high concurrent users

6. Non-functional requirements

6.1 Performance requirements

- The system shall respond to user requests within acceptable time limits under normal load conditions.

6.2 Scalability requirements

- The system shall support increasing numbers of users and bookings without performance degradation.

6.3 Security requirements

- JWT (JSON Web Tokens) shall be used for secure authentication and authorization between users and services.
- Passwords and sensitive data must be encrypted before storage.
- Role-Based Access Control (RBAC) shall be enforced to restrict access based on user roles.

6.4 Usability requirements

- The system shall provide a simple and user-friendly interface for all users.

6.5 System reliability

- The system shall operate continuously and ensure high availability for users.

6.6 Availability Requirement

- The system shall be available for users at all times with minimal downtime

7. Design & Implementation Constraints

The design and implementation of the Cinema Hub system are subject to several technical, architectural, and organizational constraints. These constraints define the boundaries within which the system must be developed and deployed.

7.1 Technology Constraints

- The backend must be developed using Spring Boot (Java).
- The frontend must be implemented using React.js.
- The system must follow a Microservices Architecture, where each service is independently deployable.
- Spring Cloud must be used for service discovery, centralized configuration, and API Gateway management in the microservices architecture.
- Spring Boot REST APIs must be used for communication between frontend and backend services as well as inter-service communication.

Impact:

These constraints enforce a standardized development stack, ensuring consistency but limiting flexibility in choosing alternative technologies.

7.2 Messaging & Concurrency Constraints (Kafka)

- The system must use Apache Kafka for handling real-time events such as seat selection and booking confirmation.
- Seat locking and release operations must be handled asynchronously using Kafka topics.
- Event consistency must be maintained across distributed services.
- Kafka enables asynchronous communication and real-time event processing between distributed services.

Impact:

Requires implementing event-driven architecture and handling eventual consistency challenges across services.

7.3 Deployment Constraints (Docker)

- All microservices must be containerized using Docker.
- The system must support deployment in a containerized environment.
- Docker ensures isolated, portable, and scalable deployment environments for all microservices.

Impact:

Ensures portability and scalability but requires container orchestration knowledge and environment configuration.

7.4 Database Constraints

- Each microservice must have its own independent database (Database per Service pattern).
- Data consistency across services must be maintained through API calls or event-driven communication.

Impact:

Prevents tight coupling but introduces complexity in data synchronization and transaction management.

7.5 Security Constraints

- Authentication must be implemented using JWT (JSON Web Tokens).
- Role-Based Access Control (RBAC) must be enforced across all services.
- Sensitive data (e.g., passwords, payment data) must be encrypted.

Impact:

Enhances security but adds complexity in token validation and inter-service communication.

7.6 Performance Constraints

- The system must support high concurrency during peak booking times.
- Seat selection operations must reflect updates in near real-time.

Impact:

Requires efficient caching, asynchronous processing, and optimized database queries.

7.7 Academic Constraints

- The system must include at least four functional modules.
- UML diagrams (Use Case, Class, Sequence, Activity) must be provided.
- The system must demonstrate usage of AOP and Microservices.

Impact:

Guides system design decisions to align with academic requirements.

7.8 Cross-Cutting Concerns Constraints

- Logging
- Security validation
- Exception handling
- Performance monitoring

Impact:

AOP improves modularity and reduces code duplication across services.

8. System API Endpoints Overview

The following section provides an overview of the main API endpoints available in the Cinema Hub system. Each microservice exposes a dedicated set of RESTful APIs responsible for handling specific business operations.

8.1 Authentication Service APIs (Port 8081)

The Authentication Service manages user authentication, authorization, and account management operations.

Method	Endpoint	Description
POST	/api/auth/login	Authenticate users and generate JWT tokens
POST	/api/auth/register	Register new user accounts
GET	/api/auth/profile	Retrieve authenticated user profile information
POST	/api/auth/logout	Logout authenticated users
GET	/api/auth/users	Retrieve all registered users (Admin access)

8.2 Movie Service APIs (Port 8082)

The Movie Service manages movies, cinema halls, and showtime-related operations.

Method	Endpoint	Description
GET	/movies	Retrieve all movies
GET	/movies/{id}	Retrieve movie details by ID
POST	/movies	Add a new movie
PUT	/movies/{id}	Update movie information
DELETE	/movies/{id}	Delete a movie

8.3 Booking Service APIs (Port 8083)

The Booking Service manages reservations, seat generation, and customer bookings.

Method	Endpoint	Description
POST	/bookings	Create a new booking
POST	/bookings/generate-seats/{showtimeId}	Generate seats for a specific showtime
GET	/bookings	Retrieve all bookings
GET	/bookings/{id}	Retrieve booking details by ID
GET	/bookings/user/{userId}	Retrieve bookings for a specific user
DELETE	/bookings/{id}	Cancel or delete a booking

8.4 Payment Service APIs (Port 8084)

The Payment Service handles payment processing and transaction management.

Method	Endpoint	Description
POST	/api/payments	Process a payment transaction
GET	/api/payments/{id}	Retrieve payment details by ID
GET	/api/payments	Retrieve all payment transactions

8.5 API Gateway (Port 8080)

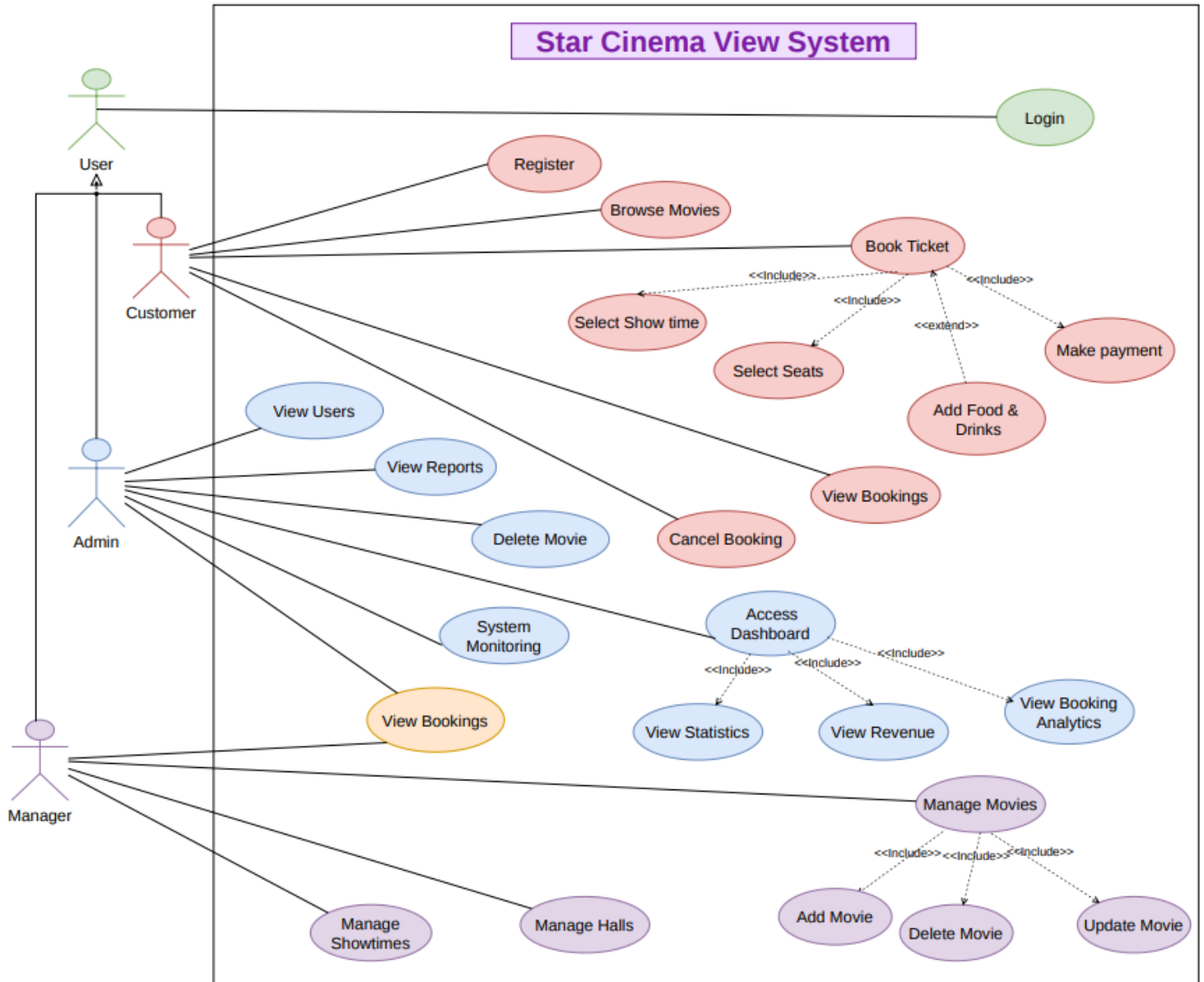
The API Gateway acts as the centralized entry point for all client requests and routes requests to the appropriate microservices.

Example Gateway Routes

Gateway Route	Forwarded Service
/api/auth/*	Auth Service
/movies/*	Movie Service
/bookings/*	Booking Service
/api/payments/*	Payment Service

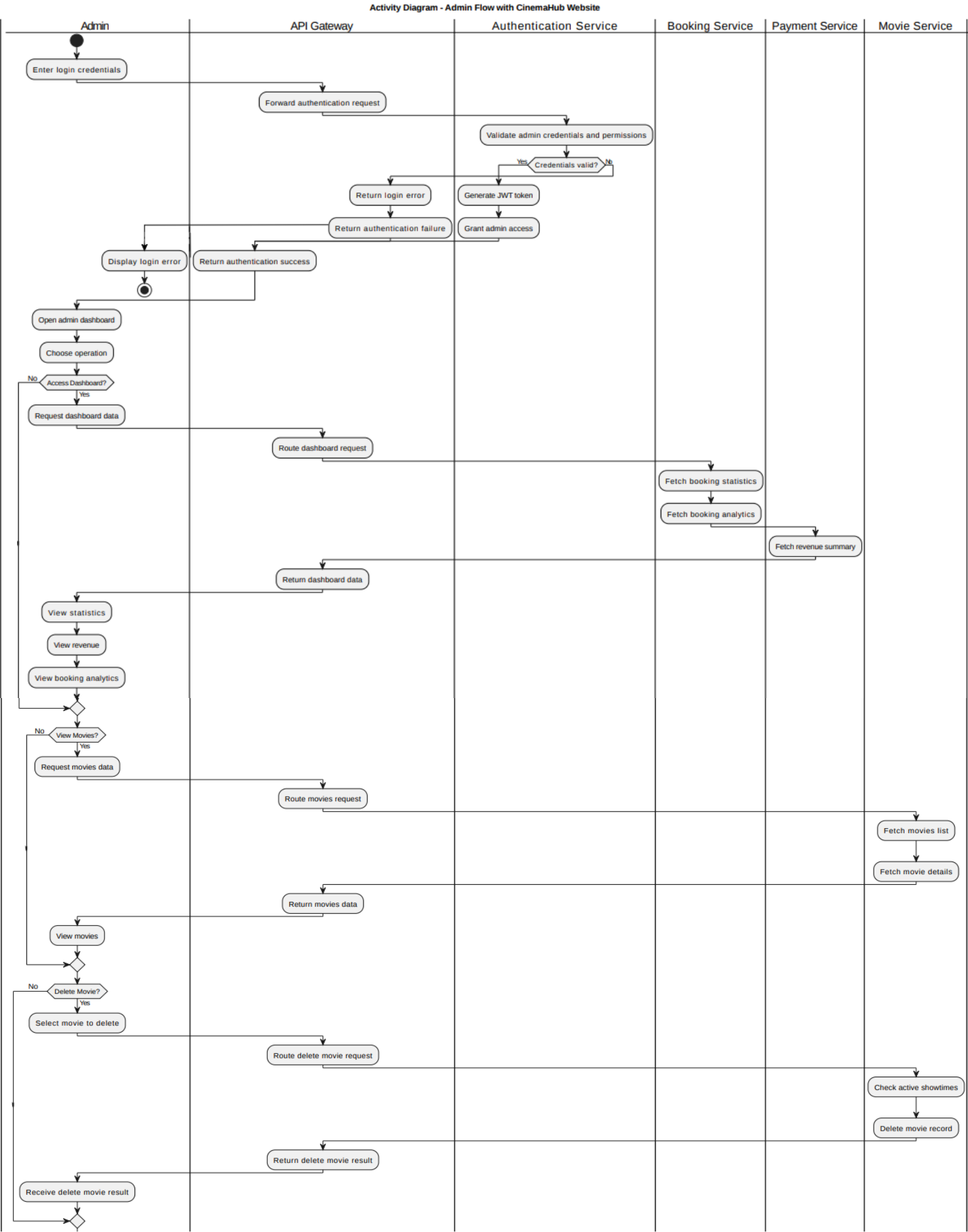
9. System Diagrams

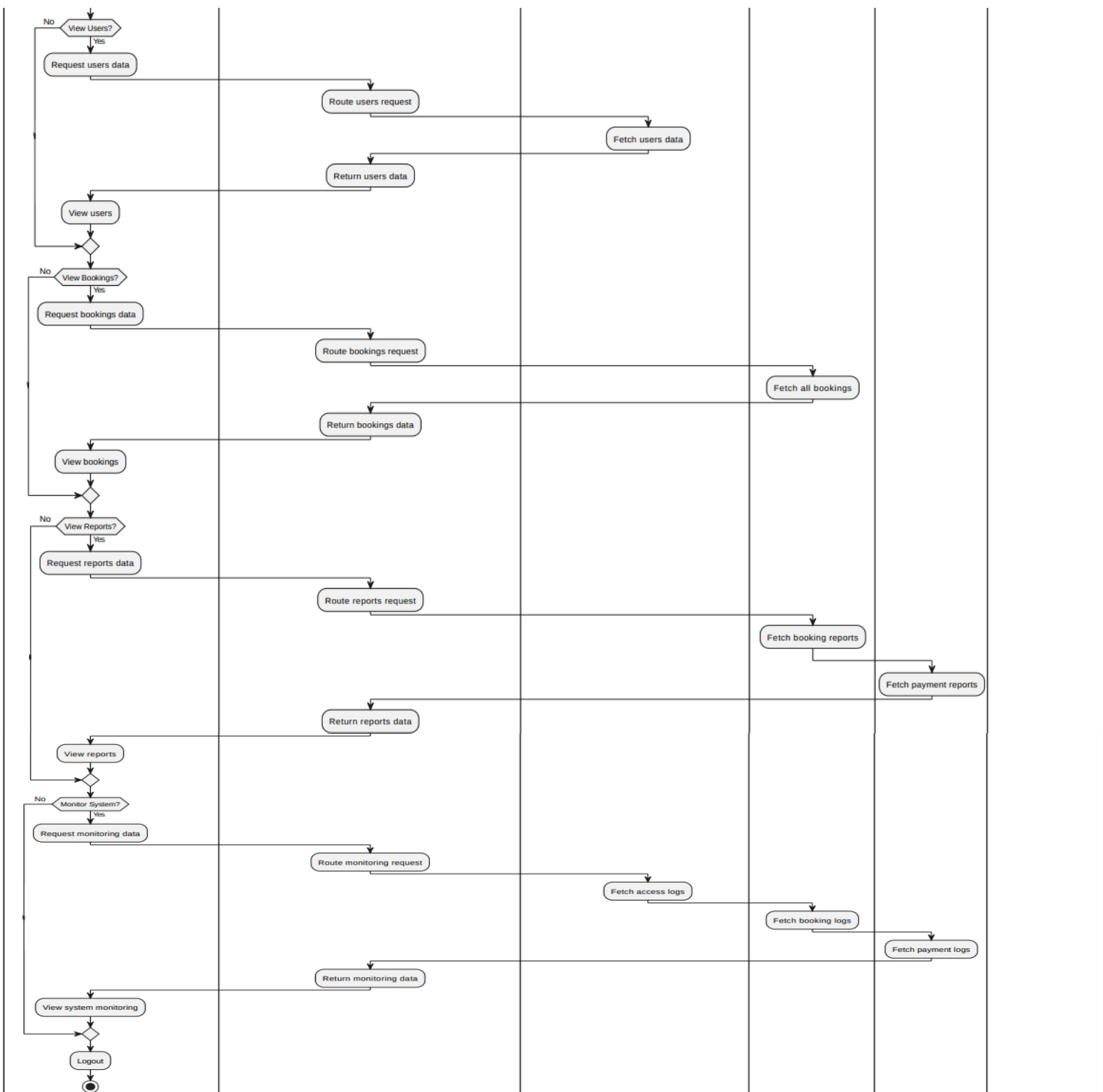
1) Use Case Diagram:



2) Activity Diagrams:

- Admin Activity Diagram:

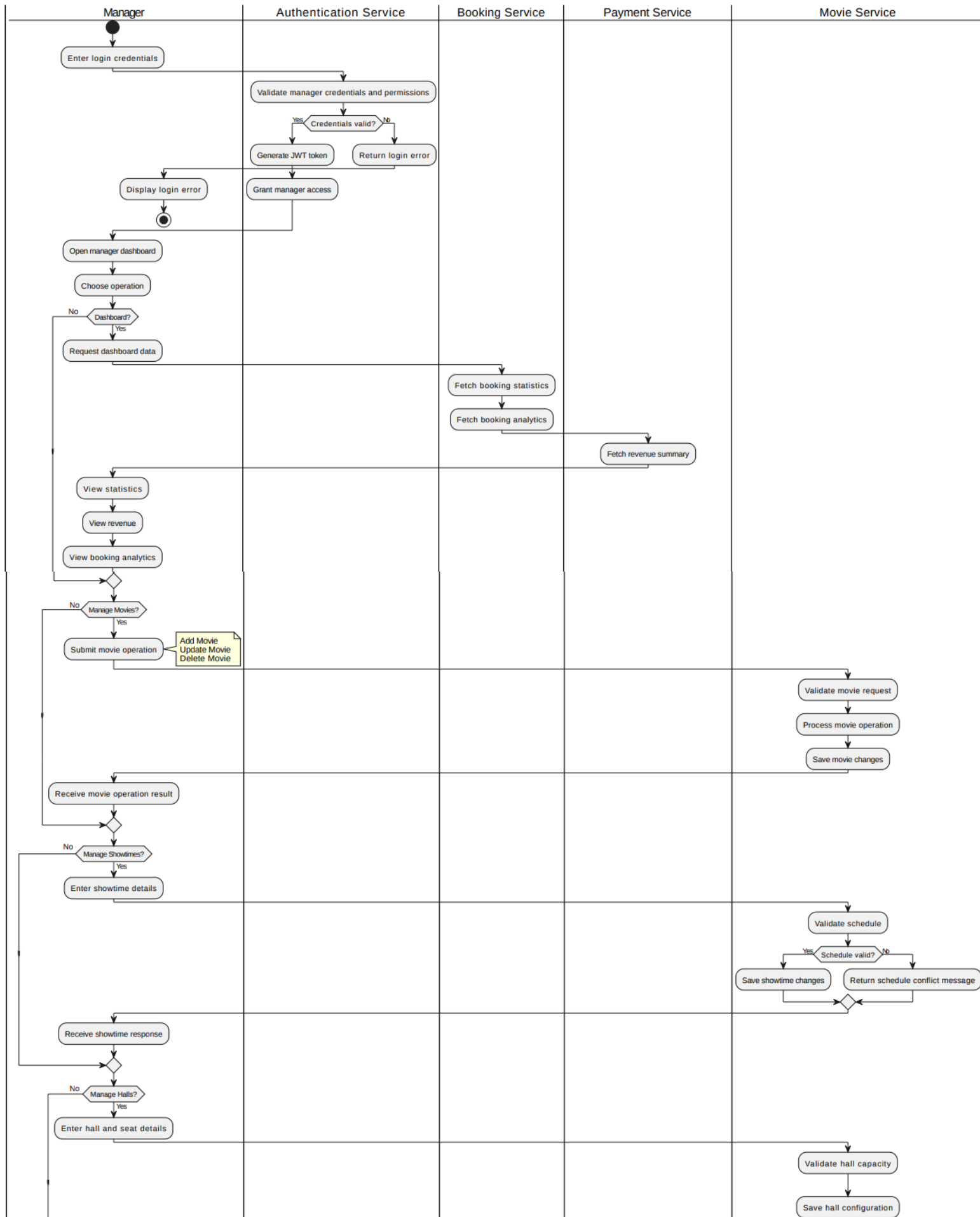


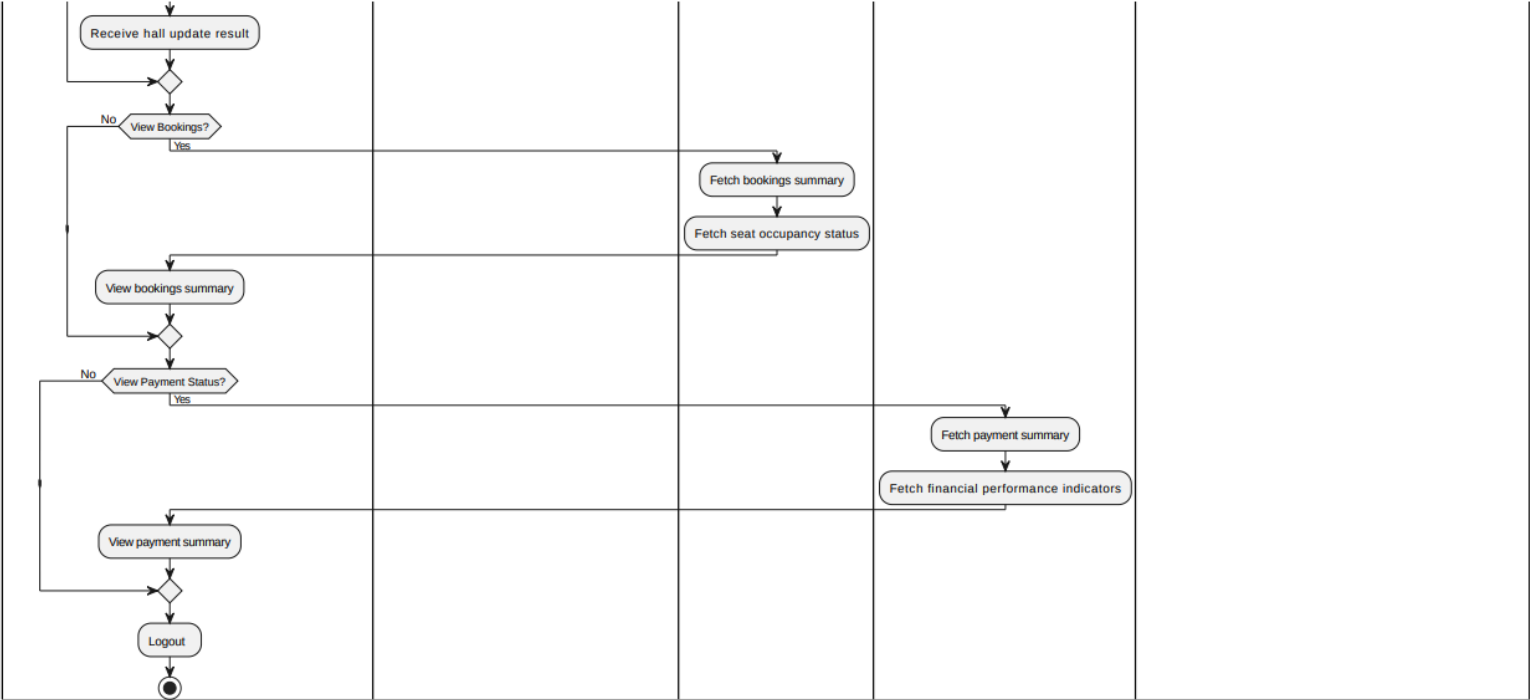


- **Manager Activity Diagram:**

Activity Diagram - Manager Flow with CinemaHub Website

Client requests are routed through API Gateway.

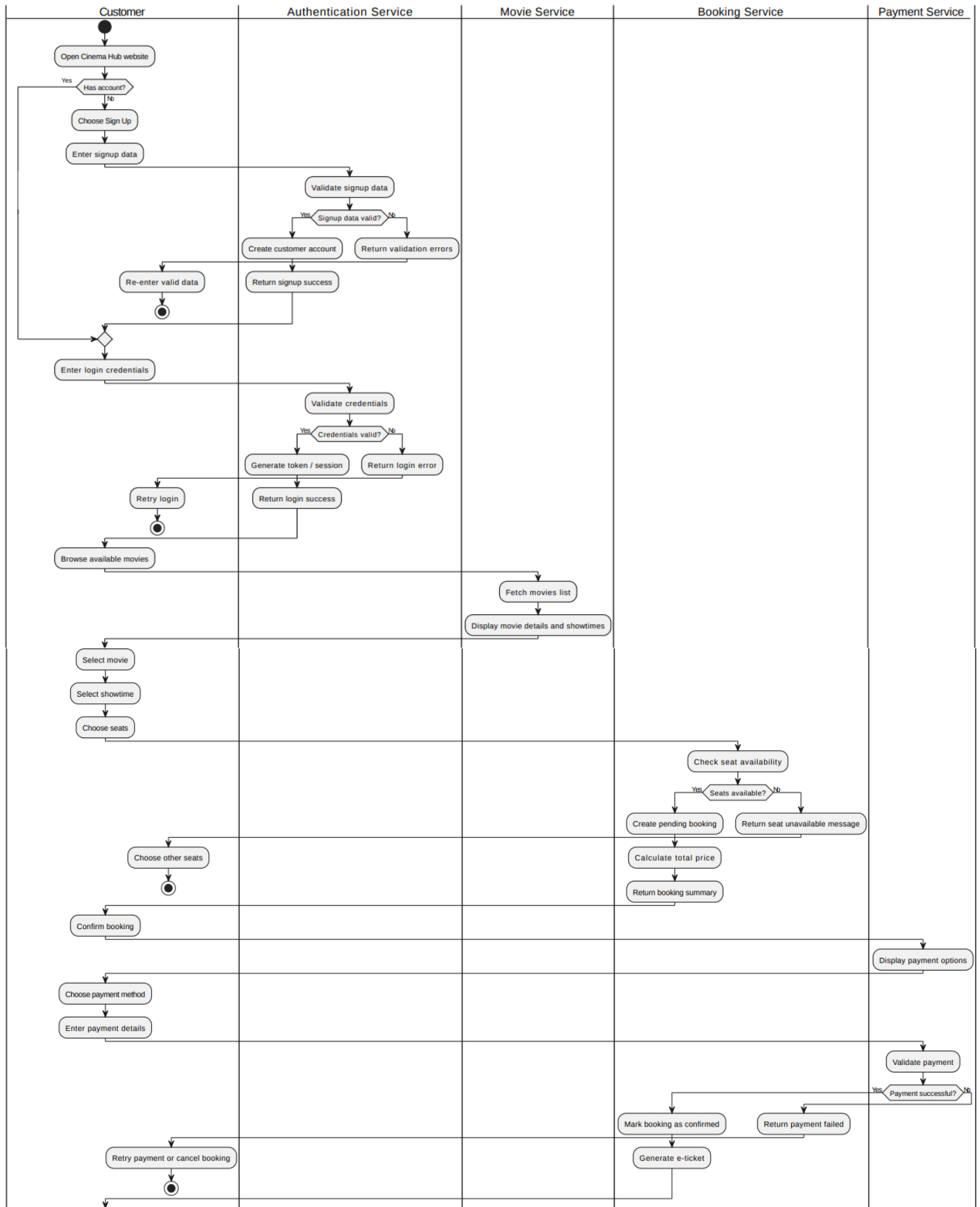


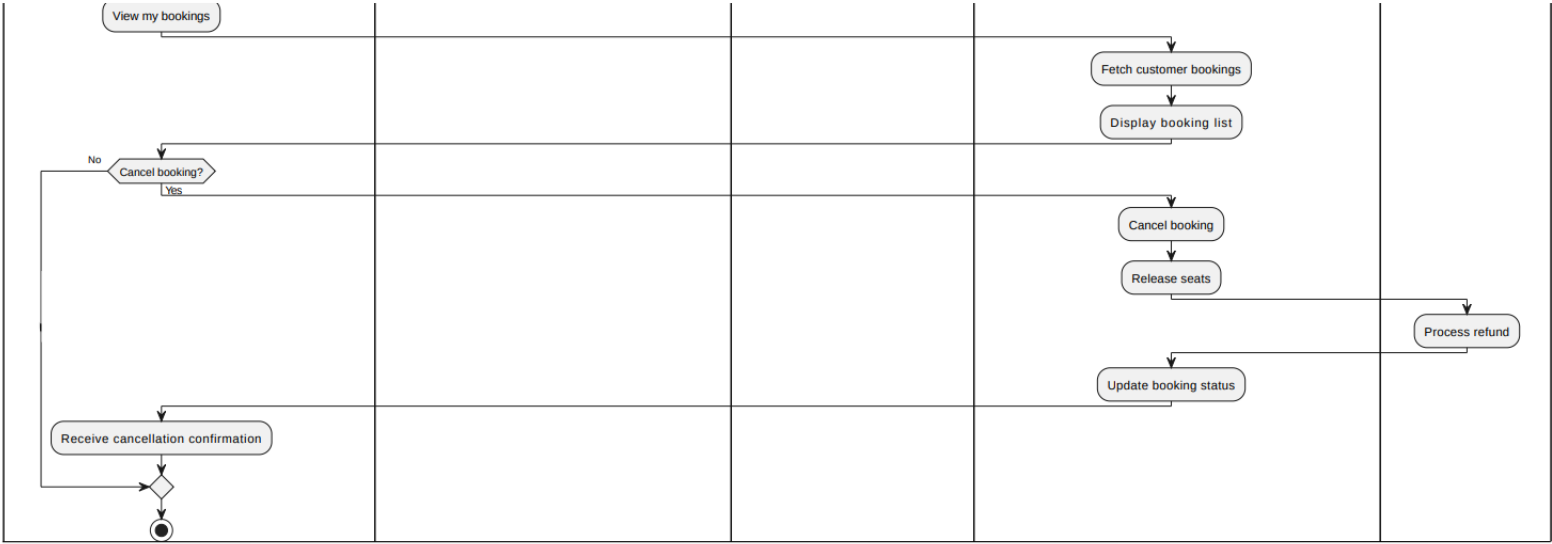


- Customer Activity Diagram:

Customer Flow with CinemaHub Website

Client requests are routed through API Gateway.

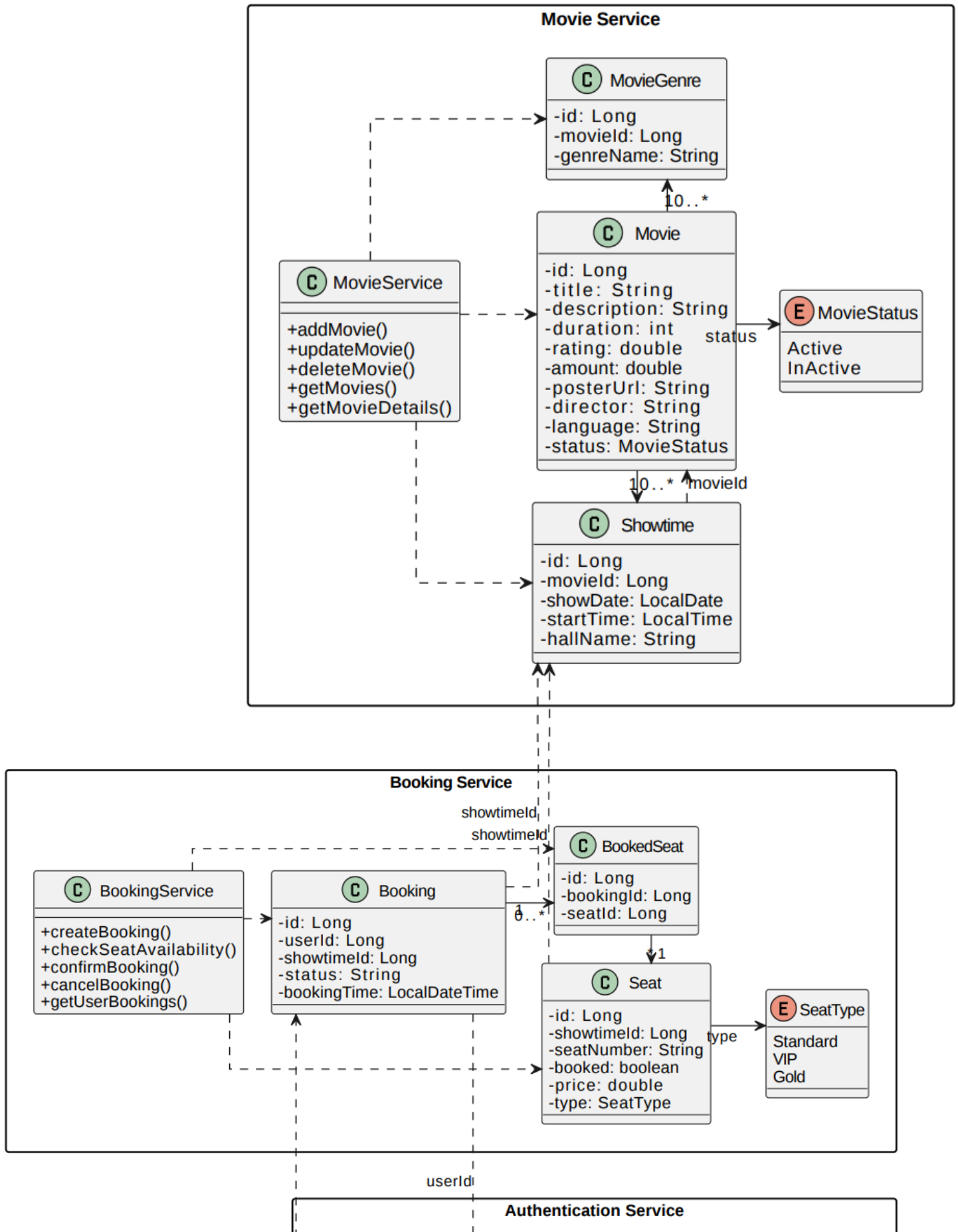


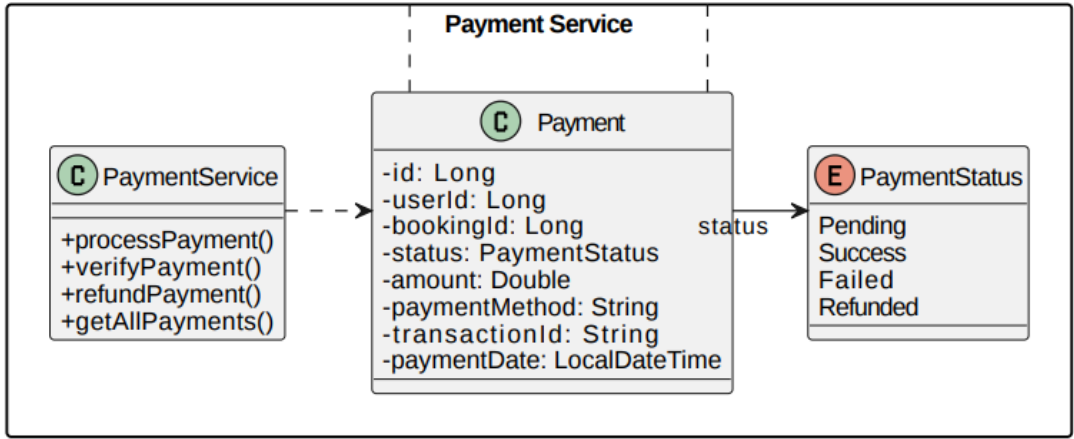
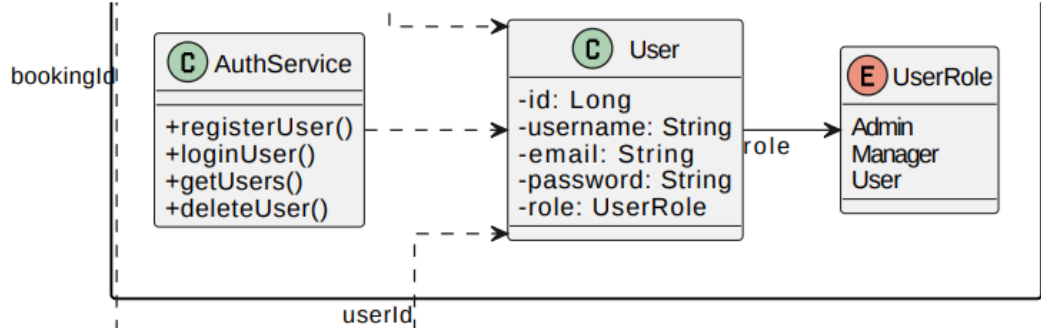


3) Class Diagrams:

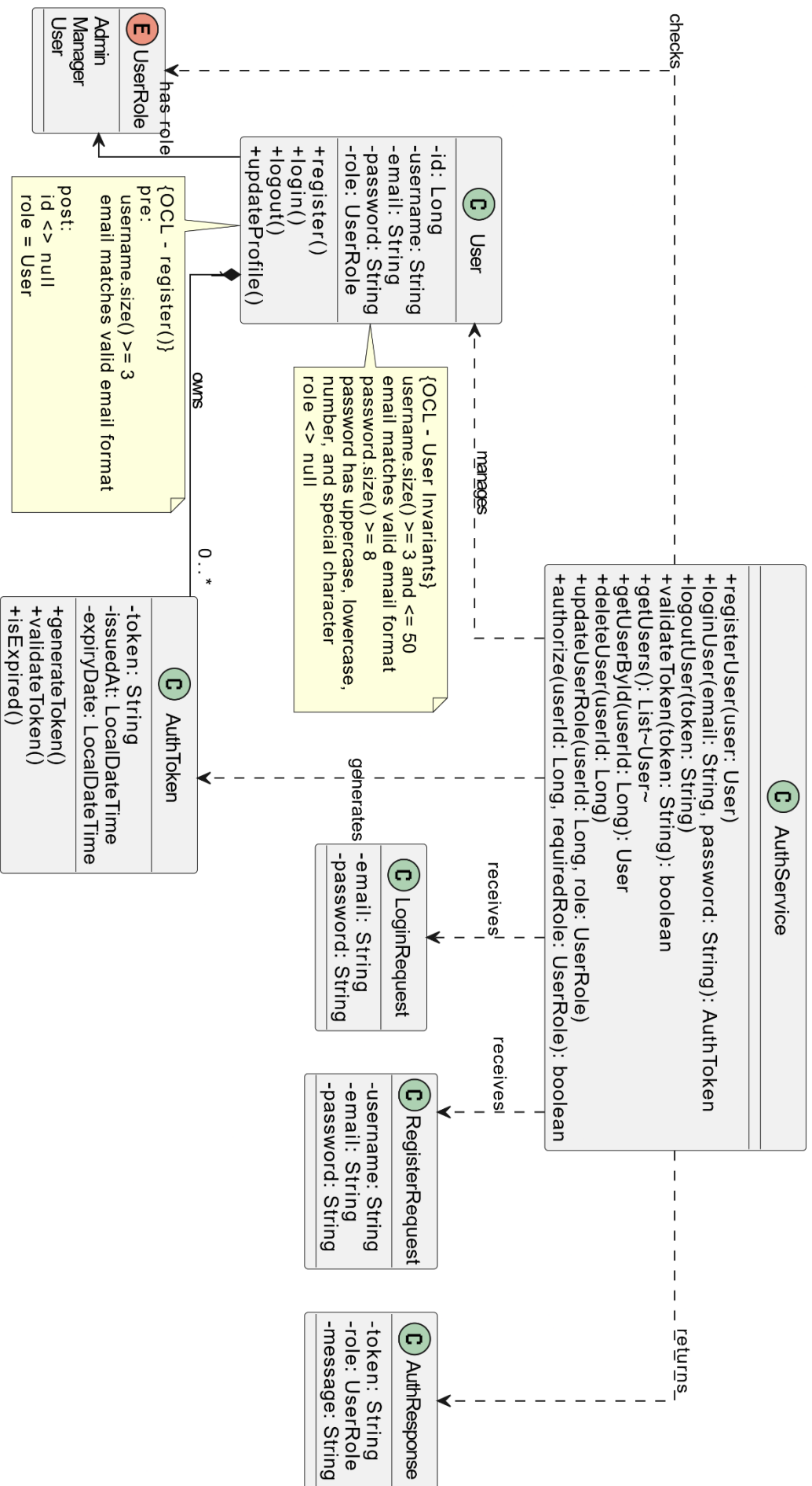
- Main Class:

Class Diagram - CinemaHub Main System





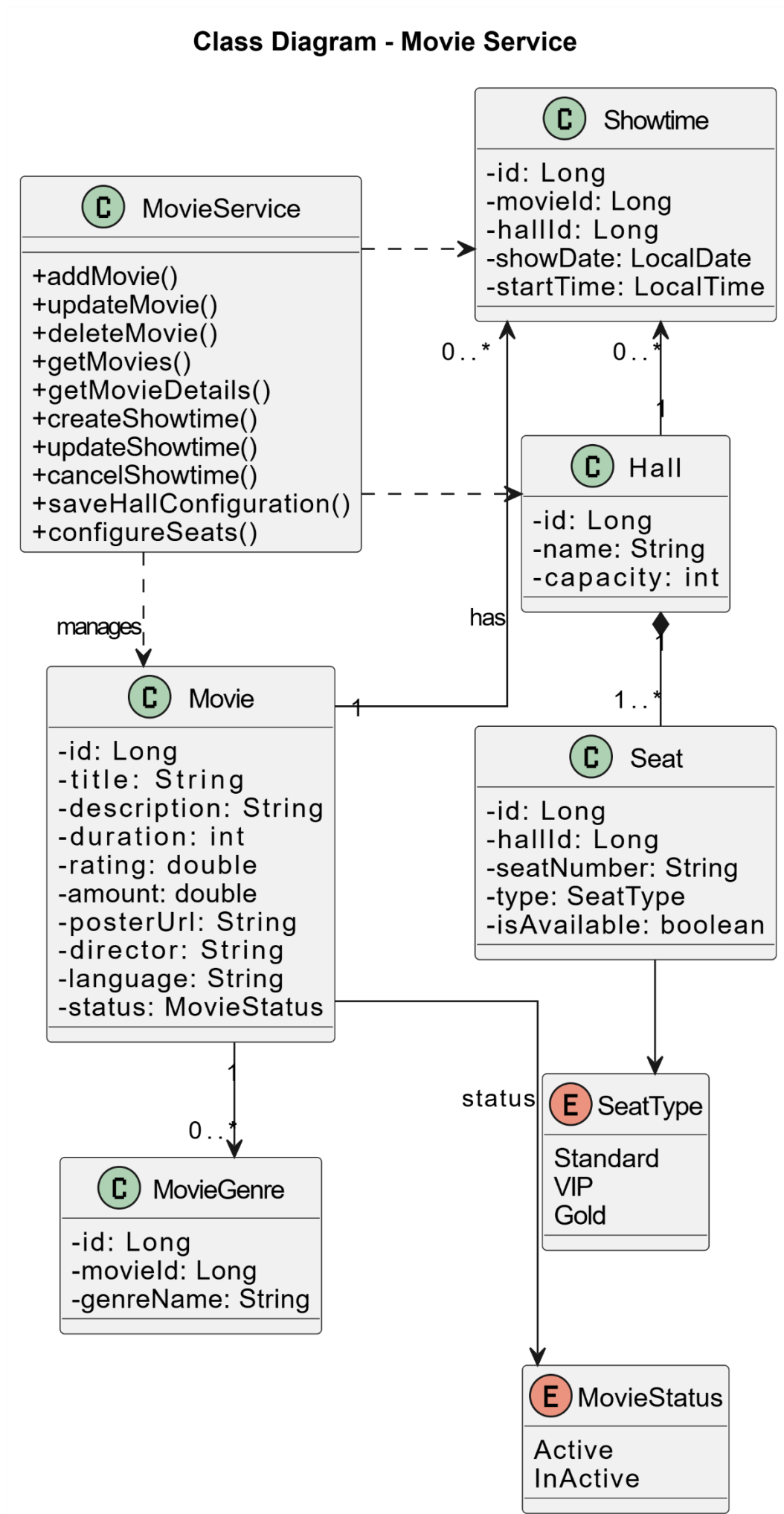
Class Diagram - Authentication / Authorization Service



Authentication Class:

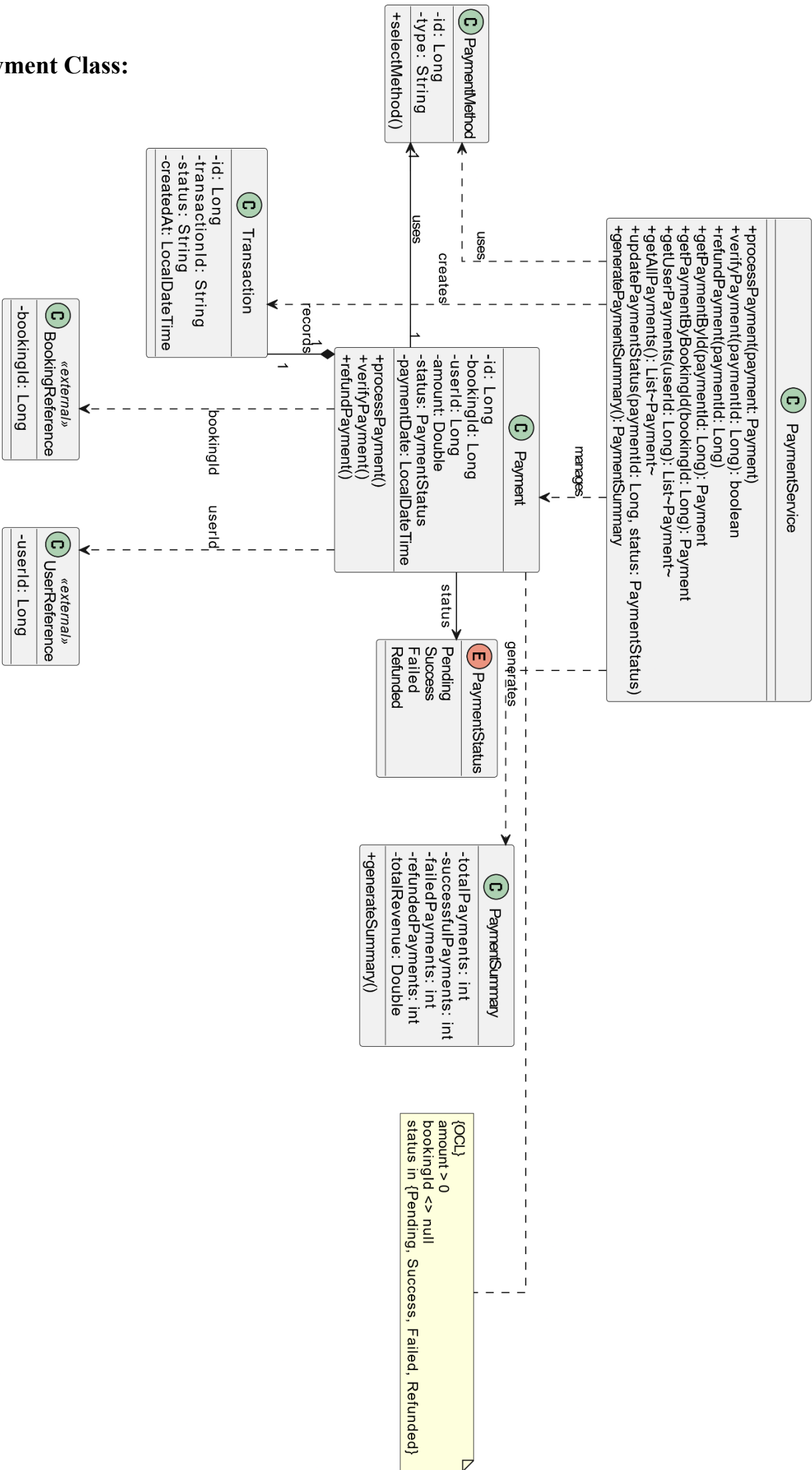
- **Movie Class:**

-

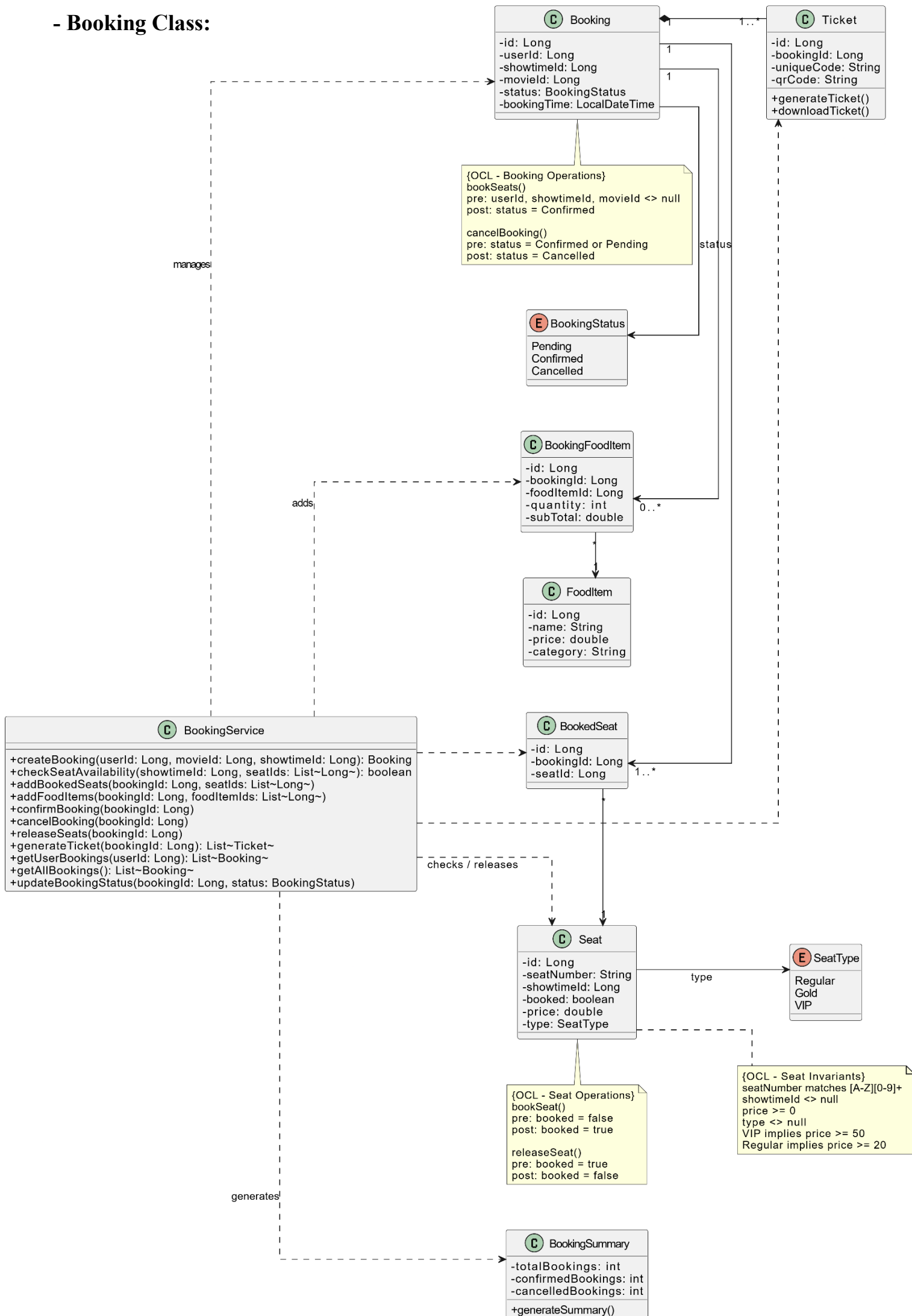


-

Class Diagram - Payment Service

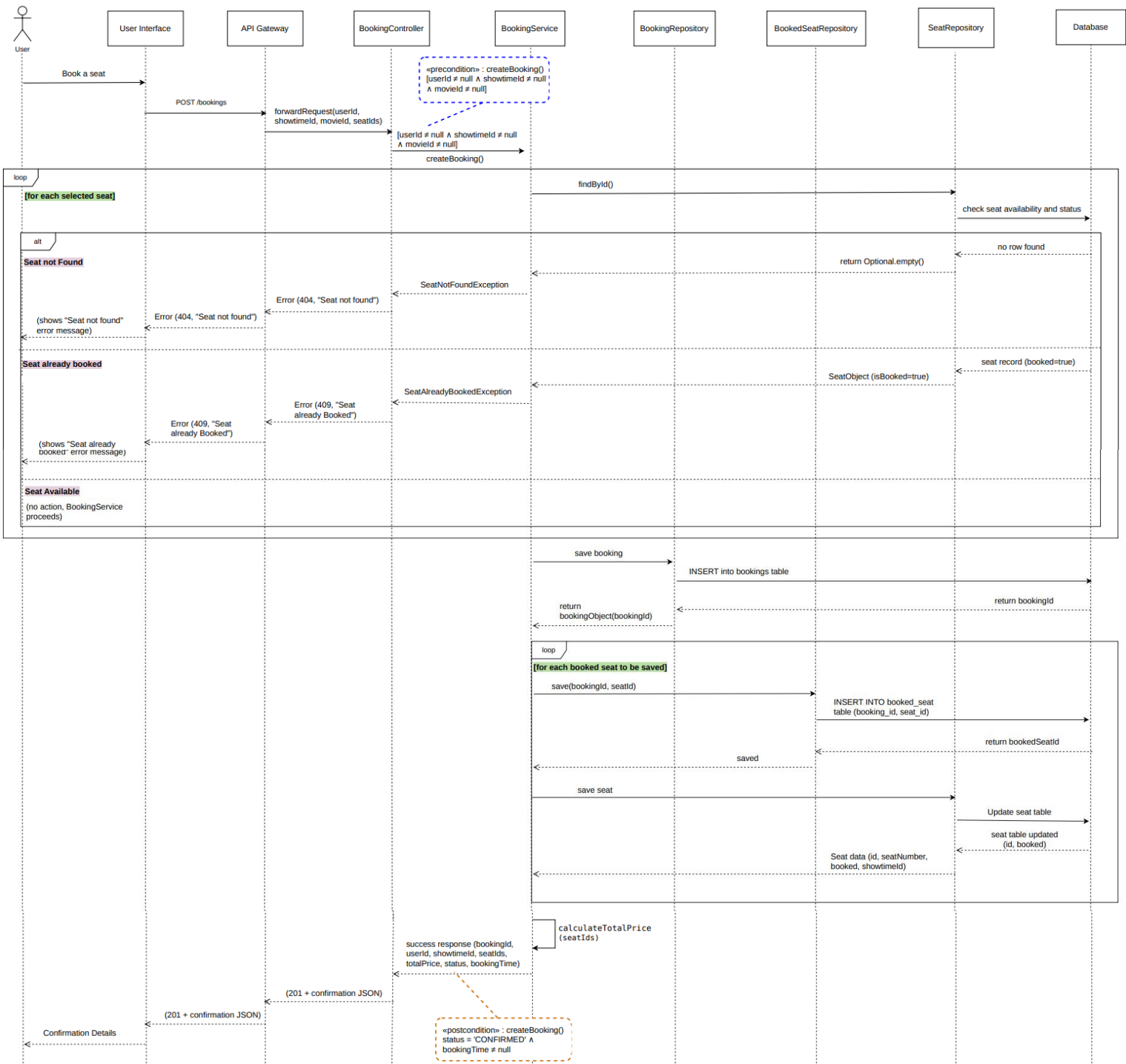


- Payment Class:

- Booking Class:

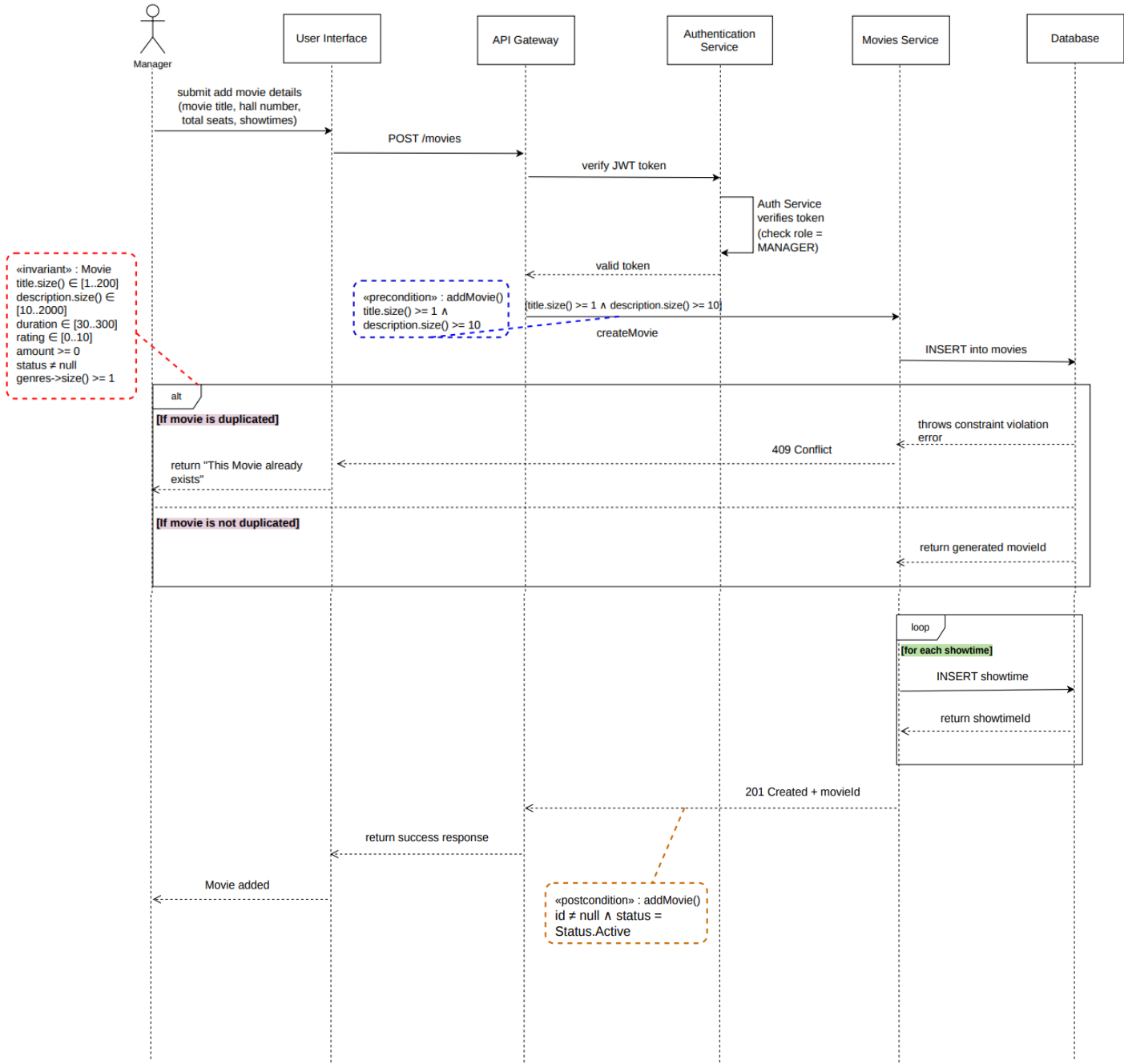
4) Sequence Diagrams:

Seat Booking Sequence Diagram (User Use Case)



- Manager Add Movie Sequence:

Add movie Sequence Diagram (Manager use case)



5) Entity Relationship Diagram (ERD):

